

Machine Learning — Additional Lecture Topics

Separate Detailed Notes for the Missing / Incomplete Topics

Prepared as a companion to the existing PCIT301 Full-Syllabus Detailed Notes PDF.

This document specifically covers the two areas that were missing/incomplete in the earlier PDF:

Precision–Recall Curve and **Preprocessing & Pipelines**. It also provides the related concepts needed to understand them properly.

Topics covered

No.	Topic
1	Precision–Recall Curve
2	Preprocessing in Machine Learning
3	StandardScaler
4	OneHotEncoder
5	SimpleImputer
6	ColumnTransformer
7	Pipeline
8	Complete Preprocessing Workflow

1. Precision–Recall Curve

The Precision–Recall (PR) curve is a classification evaluation curve that shows the trade-off between **precision** and **recall** as the classification threshold changes.

It is especially useful when the classes are **imbalanced** and the positive or minority class is important.

Key idea:

- **Precision** asks: among the observations predicted as positive, how many are actually positive?
- **Recall** asks: among all actual positive observations, how many were correctly detected?
- Changing the decision threshold changes both precision and recall.
- A lower threshold usually identifies more positive cases, which can increase recall but may also increase false positives and reduce precision.
- A higher threshold usually makes positive predictions more selective, which can increase precision but may miss more actual positives and reduce recall.

Formulae:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

How to read the PR curve:

- The horizontal axis is commonly **Recall**.
- The vertical axis is commonly **Precision**.
- Each point on the curve corresponds to a different classification threshold.
- A model that maintains high precision while achieving high recall is generally desirable.

Why PR curves are useful for imbalanced data:

Accuracy can be misleading when one class is much more common than another. For example, if 99% of observations are negative, a model that predicts every observation as negative can achieve 99% accuracy while completely failing to detect the positive class. Precision and recall focus more directly on positive-class performance.

PR Curve vs ROC Curve:

PR Curve	ROC Curve
Plots Precision vs Recall.	Plots True Positive Rate vs False Positive Rate.
Particularly informative for strongly imbalanced positive-class problems.	Useful for evaluating discrimination across thresholds.
Focuses directly on the positive predictions and positive-class coverage.	Shows the trade-off between detecting positives and generating false positives.

2. Preprocessing in Machine Learning

Data preprocessing means preparing raw data so that it is in a suitable form for machine-learning algorithms.

Preprocessing can include handling missing values, converting categorical data into numerical form, scaling numerical features, and organizing these operations into a reliable workflow.

Why preprocessing is needed:

- Real-world datasets may contain missing values.
- Categorical variables may need numerical encoding.
- Numerical features may have very different scales.
- Inconsistent preprocessing can cause errors or data leakage.
- A consistent preprocessing workflow makes model training and prediction easier to reproduce.

3. StandardScaler

StandardScaler is used to standardize numerical features so that each feature is transformed using its mean and standard deviation.

Standardized value: $z = (x - \text{mean}) / \text{standard deviation}$

After fitting on training data, the learned mean and standard deviation are used to transform the relevant data. This is important because information from the test set should not be used to fit preprocessing parameters.

Scaling is especially useful for algorithms affected by feature magnitude, such as KNN, SVM, and many optimization-based models.

4. OneHotEncoder

OneHotEncoder converts categorical values into numerical indicator columns.

Example: if a feature contains **Red**, **Blue**, **Green**, one-hot encoding can represent them with separate binary columns such as Red, Blue, and Green.

The encoded representation allows machine-learning algorithms that require numerical inputs to work with categorical features.

5. SimpleImputer

SimpleImputer is used to handle missing values in a dataset.

Common strategies include:

- **Mean:** replace missing numerical values with the mean of the feature.
- **Median:** replace missing numerical values with the median.
- **Most frequent:** replace missing values with the most frequently occurring value.
- **Constant:** replace missing values with a specified constant.

The imputer should be fitted using the training data and then applied to validation/test data using the learned replacement values.

6. ColumnTransformer

ColumnTransformer allows different preprocessing operations to be applied to different groups of columns.

For example:

- Numerical columns → imputation + StandardScaler.
- Categorical columns → imputation + OneHotEncoder.

This is useful when a dataset contains both numerical and categorical features.

7. Pipeline

A Pipeline chains preprocessing steps and a machine-learning estimator into a single workflow.

A typical pipeline can be represented as:

Raw data → Imputation → Encoding/Scaling → Model

Benefits of a pipeline:

- Keeps preprocessing and model training together.
- Applies the same transformations consistently during training and prediction.
- Helps reduce the risk of accidentally applying preprocessing differently to training and test data.
- Works well with cross-validation and hyperparameter search.
- Makes the workflow easier to reproduce and maintain.

8. Complete Preprocessing Workflow

A common machine-learning preprocessing workflow is:

1. Load the dataset.
2. Separate input features from the target variable.
3. Identify numerical and categorical columns.
4. Handle missing values with an imputation strategy.
5. Scale numerical features when required.
6. Encode categorical features into a numerical representation.
7. Combine the preprocessing operations using ColumnTransformer when different columns require different transformations.
8. Combine preprocessing and the estimator using Pipeline.
9. Split data appropriately and fit preprocessing/model components using the training data.
10. Evaluate the trained model on unseen validation/test data.

9. Important Viva Questions

Q1. What is a Precision–Recall curve?

It is a curve showing the relationship between precision and recall at different classification thresholds.

Q2. Why is a PR curve useful for imbalanced data?

Because accuracy can be misleading under strong class imbalance, while precision and recall provide direct information about positive-class performance.

Q3. What does StandardScaler do?

It standardizes numerical features using their mean and standard deviation.

Q4. Why is OneHotEncoder used?

It converts categorical values into numerical indicator features.

Q5. What does SimpleImputer do?

It replaces missing values using a chosen strategy such as mean, median, most frequent, or a constant.

Q6. What is ColumnTransformer?

It applies different preprocessing transformations to different groups of columns.

Q7. What is a Pipeline?

It chains preprocessing steps and a model into one reproducible machine-learning workflow.

Q8. Why should preprocessing be fitted only on training data?

To avoid leaking information from validation/test data into the training process.

10. Quick Revision

- PR curve → Precision vs Recall.
- Imbalanced data → accuracy may be misleading.

- StandardScaler → standardizes numerical features.
- OneHotEncoder → categorical → numerical indicator columns.
- SimpleImputer → handles missing values.
- ColumnTransformer → different transformations for different columns.
- Pipeline → preprocessing + model in one workflow.
- Avoid data leakage → fit preprocessing on training data, then transform unseen data.